

# NossoBolso

Finanças Compartilhadas: aplicação CRUD com cinco visões de usuário

Relatório do projeto e do processo de desenvolvimento com agentes de código

## Equipe

Heitor Gabriel Lucena Albuquerque

Victória Oliveira Estrela

Jamilly Melo Fernandes

Docente: Prof. Andrei Formiga

Julho de 2026

Repositório: [github.com/hglucena/NossoBolso](https://github.com/hglucena/NossoBolso)

## 1. A aplicação

O NossoBolso é uma aplicação web para controle de finanças pessoais e compartilhadas. Cada tipo de usuário enxerga uma visão diferente do sistema, com painel e rotina de uso próprios. Uma pessoa cuida das próprias contas; um grupo (uma família ou uma república) mantém um orçamento comum com divisão de despesas; um dependente acompanha apenas a sua mesada; um consultor financeiro acompanha a carteira de clientes que o autorizaram; um administrador cuida da plataforma sem acesso às finanças de ninguém.

| Visão         | O que acessa                                                                                                                                                                                                    |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Membro        | Contas com saldo calculado, transações, categorias, orçamentos, contas a pagar e metas de economia. Importa extrato CSV e autoriza ou revoga consultores. Vê os grupos de que participa, mas não os administra. |
| Gestor        | Painel do grupo: membros, orçamento (previsto contra realizado), despesas divididas, "quem deve a quem" e mesadas, que ele cria e recarrega.                                                                    |
| Dependente    | Apenas a própria mesada e os gastos pessoais. Gasto acima do saldo é bloqueado. A mesada recarrega automaticamente por período e o dependente pode criar metas próprias com aportes saindo dela.                |
| Consultor     | Carteira de clientes autorizados, em modo leitura. Cria recomendações quando o nível de autorização é "comentar".                                                                                               |
| Administrador | Gestão de usuários e categorias padrão. Nunca acessa finanças alheias.                                                                                                                                          |

O backend é Python 3.11 com Django 4.2 e Django REST Framework, servido por gunicorn com whitenoise; o banco é PostgreSQL 15; o frontend é React (Vite) com Tailwind CSS e Recharts, servido por nginx; a autenticação usa o TokenAuthentication do DRF. Tudo roda em containers Docker orquestrados por um `docker-compose.yml` com três serviços: `db`, `backend` e `frontend`.

A decisão de modelagem que sustenta as cinco visões: o papel de sistema do usuário é apenas `comum` ou `admin`. Os demais papéis são definidos por relacionamento, e não no cadastro. Ser responsável ou dependente é um papel dentro de um grupo, guardado em `MembroGrupo.papel_no_grupo`; ser consultor de alguém é uma autorização, guardada em `AutorizacaoConsultor`. A mesma pessoa pode ser responsável de um grupo, membro de outro e cliente de um consultor sem precisar de vários tipos de conta.

O modelo de dados tem 13 entidades: `Usuario`, `Conta`, `Categoria`, `Transacao`, `Grupo`, `MembroGrupo`, `Orcamento` e `DivisaoDespesa` no MVP; `Mesada`, `AutorizacaoConsultor`, `ContaAPagar` e `Recomendacao` na evolução; e `MetaEconomia`, adicionada na fase de extras. As validações de integridade (unicidade, soma das partes de uma divisão, conta e categoria pertencentes ao dono da transação) ficam nos próprios models. O diagrama entidade-relacionamento completo está em `docs/arquitetura.md`.

## 2. Como o desenvolvimento foi conduzido

---

O projeto foi construído inteiramente com agentes de código, em três fases.

Na primeira fase, de planejamento, explicamos o projeto ao Claude a partir do plano da atividade (versionado em `docs/plano.md`) e pedimos que ele gerasse uma sequência de prompts pronta para ser executada por outro agente. O resultado, salvo em `prompts-claude-code-financas-compartilhadas.md`, tem doze prompts em ordem de dependência: contexto e arquitetura; infraestrutura Docker; modelo de dados; autenticação; API CRUD com permissões; regras de negócio do grupo; testes automáticos; frontend; empacotamento; as visões Dependente e Consultor; e uma revisão final.

Na segunda fase, de execução, colamos os prompts um por vez no DeepSeek V4 Pro. Depois de cada etapa rodamos uma bateria de testes (a suíte automatizada e uma verificação manual do resultado) antes de commitar e avançar. Cada etapa foi registrada no `DEVLOG.md`: o que o agente acertou de primeira, onde errou e as decisões tomadas. O `docs/checklist.md` serviu de controle de progresso, e todas as tarefas do MVP e da evolução terminaram concluídas.

Na terceira fase, de fechamento, o quadro mudou. Ao usar a aplicação de ponta a ponta, o resultado do DeepSeek se mostrou cheio de bugs, sobretudo furos de permissão e telas exibindo dados errados. Pedimos então ao Claude uma varredura completa do projeto. Ele identificou os problemas (entre eles um de segurança), corrigiu todos e ampliou a suíte de testes. Depois dessa varredura a aplicação ficou estável em todos os fluxos. A seção 4 detalha o que foi encontrado.

A disciplina de trabalho foi a mesma do começo ao fim: colar um prompt, revisar o diff gerado, rodar o resultado, registrar no DEVLOG, commitar e só então avançar. Em nenhum momento aceitamos código sem executar.

### 2.1 O que os agentes acertaram de primeira

A infraestrutura subiu na primeira execução: Docker Compose com healthcheck do PostgreSQL e `depends_on: service_healthy`, Django e DRF conectados ao banco por variável de ambiente, endpoint `/api/health/` respondendo. O modelo de dados também veio correto de primeira, incluindo a decisão dos papéis por relacionamento. O scaffold do frontend (Vite, Tailwind, Recharts, login, roteamento por papel e três painéis) funcionou sem nenhum ajuste manual, o que nos surpreendeu, já que React era a tecnologia em que o grupo tinha menos experiência. O empacotamento (Dockerfiles de produção e o comando `seed_demo`) tampouco precisou de correção.

### 2.2 Fricções técnicas

Quatro problemas tomaram mais tempo do que o esperado. O login por e-mail com User customizado exigiu um `UsuarioManager` próprio, porque o `UserManager` padrão do Django ainda espera `username`. O campo `divisoes` (uma lista de objetos aninhada no serializer de transação) chega serializado de forma diferente no `APITestCase` e em requests reais, e a saída foi ler `self.initial_data` diretamente. Um teste media o tamanho do dict de paginação em vez da lista vazia por causa de um `len(results or data)`, já que lista vazia é falsy em Python; a correção foi usar `response.data.get("count", 0)`. Por

fim, trocar o `runserver` por `gunicorn` em produção quebraria o CSS do Django Admin, o que foi resolvido com `whitenoise` e `collectstatic` no start do container.

### 3. Como o plano de testes funcionou na prática

O código de permissões gerado pelos agentes parecia correto na leitura. Foram os testes automatizados que revelaram os furos. A suíte foi escrita por critério de aceitação, com um teste para cada regra do §5 do plano, e cresceu junto com o projeto:

| Marco                | Suíte                | Observação                                           |
|----------------------|----------------------|------------------------------------------------------|
| Prompt 6 (MVP)       | 67 testes, 0 falhas  | autenticação, permissões, CRUD, integridade, negócio |
| Prompt 11 (evolução) | 99 testes, 0 falhas  | mais Dependente (17) e Consultor (15)                |
| Fase de extras       | 142 testes, 0 falhas | contas a pagar, metas, CSV, mesadas, segurança       |
| Saldo vivo           | 150 testes, 0 falhas | pagamento com lançamento, metas do dependente        |
| Entrega final        | 165 testes, 0 falhas | após a varredura final de bugs                       |

Os testes de permissão pegaram bugs reais logo no MVP. Qualquer membro conseguia editar ou deletar o grupo, porque faltava restringir a escrita ao responsável. Um usuário conseguia editar a categoria de outro se soubesse o ID. O administrador estava bloqueado de criar categorias padrão, porque a barreira `NaoAdmin` era ampla demais. Qualquer membro conseguia criar orçamento de grupo sem ser gestor. Mais tarde, os testes da visão Dependente mostraram que o dependente ainda via os membros e o "quem deve a quem" do grupo, e a varredura final descobriu que um admin criado pela API não recebia `is_staff` e portanto não conseguiria entrar no Django Admin.

O padrão se repetiu ao longo do projeto: o agente entrega rápido um esqueleto quase todo correto, e o que sobra de errado está concentrado nas regras de acesso, justamente a parte mais sensível. Esses furos não aparecem em uso casual, porque exigem simular um segundo usuário mal-intencionado. Todos os critérios do §5 do plano terminaram cobertos por teste, com a correspondência entre critério e teste documentada no DEVLOG.

Um dos bugs mais instrutivos só apareceu quando os dados de demonstração ficaram realistas. Com o seed completo (duas famílias, uma república, seis meses de histórico), o gestor João caía no painel de dependente e via a mesada do filho como se fosse dele. O frontend classificava como dependente qualquer usuário cuja consulta a `/api/mesadas/` retornasse resultados, e o gestor também enxerga mesadas, as dos dependentes do grupo. Como o seed antigo não tinha mesada nenhuma, a heurística nunca havia falhado. Ficou a lição de que dados de demonstração plausíveis funcionam como um teste de integração manual: expõem suposições que dados pobres deixam escondidas.

### 4. A varredura final de bugs

Ao término dos prompts a aplicação aparentava funcionar, mas o uso real mostrou outra coisa. A maior parte dos problemas era da categoria mais difícil de notar testando na mão: nada quebrava com erro, as

telas apenas mostravam números errados ou fluxos sem saída. A varredura pedida ao Claude encontrou sete problemas.

O mais grave era de segurança. O endpoint de autorização de consultor salvava o payload recebido sem validar o campo `cliente`, de modo que qualquer usuário podia enviar `{consultor: eu, cliente: vítima}` e ganhar acesso de leitura às finanças de um terceiro. A correção fixa o cliente como o usuário logado e valida duplicidade e autoconsultoria.

Dois problemas produziam dados errados. O `orcamento_resumo` ignorava o período e somava despesas de todos os meses contra um limite mensal; com seis meses de histórico, o painel de previsto contra realizado mostrava estouros falsos. E o painel do gestor buscava a primeira página geral de transações (20 itens) e filtrava no cliente, truncando a lista do grupo; passou a usar o filtro do backend.

Dois eram de regra de negócio na interface. A divisão em partes iguais incluía os dependentes no rateio, quando dependente tem mesada e não divide aluguel. E a divisão manual era um stub: o modal enviava `divisoes: []` e sugeria usar a API; a tela prometida no plano, com um campo de valor por membro e a soma parcial exibida, foi implementada nessa etapa.

Os dois últimos eram de navegação. As sub-abas do painel do gestor nunca renderizavam, por uma condição de exibição contraditória no React, e o painel nem tinha link na barra de navegação: só chegava lá quem digitasse a URL. Havia ainda um resquício do endurecimento de segurança: o botão de autorizar consultor continuava no painel do próprio consultor, onde, com a nova regra, autorizava um consultor para ele mesmo. O botão foi removido e o fluxo correto vive no painel do membro.

Com as correções, a suíte chegou aos 165 testes sem falhas e a aplicação passou a se comportar corretamente nos fluxos das cinco visões.

## 5. Validação com o cliente

---

O plano previa validar os critérios de aceitação com um cliente real, uma família ou uma república. Usamos como cliente de referência o cenário da Família Silva dos dados de demonstração e percorremos na interface os fluxos que uma família de verdade executaria, com a equipe alternando entre os papéis. Verificamos todos os critérios da versão mínima: cadastro e login com distinção de administrador, finanças pessoais isoladas por usuário, despesa de grupo com a soma das partes fechando o total, "quem deve a quem" coerente com os lançamentos, mesada bloqueando estouro, consultor em modo leitura e administrador sem acesso a finanças.

A sessão também rendeu ajustes de usabilidade, como aconteceria com um cliente de verdade. "Não sabia como chegar no painel do grupo" levou à adição do link Grupos na navegação, que antes não existia. "Quando o gasto do Pedro era bloqueado, não aparecia o motivo" levou à exibição das mensagens de erro da API nas telas. "Queria autorizar o consultor pelo e-mail, não por número de ID" levou o endpoint a aceitar `consultor_email`. Fica como ressalva a validação com uma família externa ao grupo, recomendada antes de um uso real.

## 6. Empacotamento e entrega

---

O requisito de containerização está atendido. O repositório tem dois Dockerfiles e um `docker-compose.yml` que sobe a aplicação inteira do zero. O do backend parte de `python:3.11-slim`, instala as dependências e, no start do container, roda `migrate` e `collectstatic` antes de servir com gunicorn (3 workers), uma configuração de produção e não de desenvolvimento. O do frontend usa build em dois estágios: `node:18-alpine` compila o Vite e `nginx:alpine` serve os estáticos com proxy reverso para a API. O compose orquestra o PostgreSQL 15 (com healthcheck e volume persistente), o backend, que aguarda o banco ficar saudável, e o frontend.

```
docker compose up -d --build
docker compose exec backend python manage.py seed_demo
docker compose exec backend python manage.py test
```

O `seed_demo` cria um cenário completo e idempotente: duas famílias, uma república com três moradores, dois consultores com carteiras de clientes, seis meses de histórico de transações para os gráficos, mesadas com gastos, contas a pagar e metas. A aplicação fica em `http://localhost:5173` e a API em `http://localhost:8000`.

## 7. Balanço do processo

---

A entrega final tem as cinco visões funcionais de ponta a ponta, cada uma com painel próprio no React; todos os critérios do §5 do plano cobertos por teste automatizado; os extras da fase de evolução implementados (contas a pagar com lembretes por e-mail, metas de economia com aportes, importação de extrato CSV, recarga de mesada automática e manual, filtros de transações e o saldo calculado a partir das transações, que por decisão de projeto pode ficar negativo); e 165 testes executáveis com um comando.

O ganho de velocidade com os agentes foi enorme no scaffold e nas camadas repetitivas: serializers, viewsets e telas CRUD saíram em minutos. Separar o planejamento (o Claude gerando a sequência de prompts a partir do plano da atividade) da execução (o DeepSeek implementando um prompt por vez) deu estrutura ao trabalho, porque cada etapa tinha objetivo, critério de verificação e commit próprios.

O risco também ficou claro. O código gerado parecia correto na leitura, e os defeitos estavam exatamente onde mais importava: nas regras de acesso e nos cálculos exibidos ao usuário. Sem a bateria de testes após cada etapa e sem a varredura final, um bug de segurança real e vários painéis com dados errados teriam ido para a entrega, porque nenhum deles aparecia em uso casual.

Do que praticamos, o que de fato funcionou: pedir o teste junto com a funcionalidade, já que os testes escritos por critério de aceitação pegaram os bugs do próprio agente que os escreveu; manter o DEVLOG como diário de bordo obrigatório, que virou a matéria-prima deste relatório; nunca aceitar código sem rodar; fechar com uma revisão dirigida por um segundo agente, pois a varredura do Claude sobre o código do DeepSeek achou o que o nosso uso manual não achou; e investir em dados de demonstração realistas. A divisão de trabalho que emergiu ao longo do semestre foi simples: nós

definimos escopo e critérios e revisamos os diffs, os agentes implementam e documentam, e os testes arbitram.